getint関数について

準備

- この資料を含むディレクトリ（おそらくtest_getint）を（USBメモリではなく）PCのホームディレクトリ上にコピーする。
- vscode
  - test_getintディレクトリをvscodeで開く。
    code test_getint
  - main.cおよびgetint.hを(vscodeで)開く。
- 端末
  - test_getintディレクトリ内でコンパイル・実行する

```
cd test_getint
gcc -Wall -DLINUX -o main100 getint.c main.c
./main100 -a10 -b20 -b0xff
```

  - なぜこのような出力になるのか考える。

main関数の仮引数について

```
int main(int argc, char ** argv){
 printf("argc = %d¥n", argc);
 for(i = 0; i < argc; i++){
   printf("%d: %s¥n", i, argv[i]);
 }
}
```

コマンド

```
./main100 -a10 -b20 -b0xff
```

出力

argc = 4

0: ./main100

1: -a10

2: -b20

3: -b0xff

getint関数の引数について

```
getint(
      argc,
      argv,
      "-a",
      &val
);
```

→ ①文字列の個数
→ ②文字列（へのポインタ）の配列の先頭アドレス
→ オプション名（文字列の先頭）
→ オプション値の代入先

```
①: 4
②argv[0]: "./main100"
  argv[1]: "-a10"
  argv[2]: "-b20"
  argv[3]: "-b0xff"
```

val ← 10
返却値：0

getint関数の引数について

```
getint(
      argc,
      argv,
      "-b",
      &val
);
```

→ ①文字列の個数

→ ②文字列（へのポインタ）の配列の先頭アドレス

→ オプション名（文字列の先頭）

→ オプション値の代入先

```
①: 4
②argv[0]: "./main100"
  argv[1]: "-a10"
  argv[2]: "-b20"
  argv[3]: "-b0xff"
```

val ← 20
val ← **255**
返却値：0

getint関数の引数について

```
getint(
      argc,    → ①文字列の個数
      argv,    → ②文字列（へのポインタ）の配列の先頭アドレス
    ”./main",→ オプション名（文字列の先頭）
      &val     → オプション値の代入先
);
```

```
①: 4
②argv[0]: "./main100"
↓ argv[1]: "-a10"
  argv[2]: "-b20"
  argv[3]: "-b0xff"
```

val ← 100
返却値：0

getint関数の引数について

```
getint(
      argc-1,
      argv+2,
     ”./main",
      &val
);
```

→ ①文字列の個数

→ ②文字列（へのポインタ）の配列の先頭アドレス

→ オプション名（文字列の先頭）

→ オプション値の代入先

```
①: 3
  argv[0]: "./main100"
②argv[1]: "-a10"
↓ argv[2]: "-b20"
  argv[3]: "-b0xff"
```

valへの代入は無し
返却値：1

作業

- getintの第1引数をargc+1に、第2引数をargv-1に修正して実行結果の違いを確認せよ。
- 課題に入力せよ
- 確認が終わったら、vscodeを終了し、test_getintディレクトリを自分のUSBメモリ内にカット＆ペースト（移動）せよ。